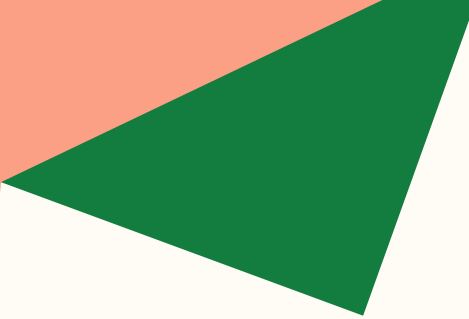
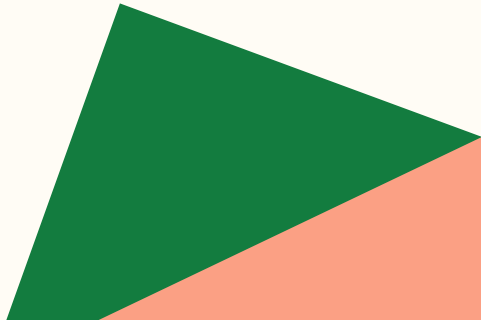




Introduction to Hadoop

Mahfuzur Rahman Emon

Lecturer, IICT, SUST



Introduction to Big Data

- The digital transformation across industries generates massive amounts of data.
- Big Data refers to data sets that are so large or complex that traditional data processing software cannot manage them effectively.
- Examples: social media interactions, online transactions, sensor data, multimedia content.

Big Data

Volume: The amount of data generated every second (e.g., terabytes to zettabytes).

Velocity: The speed at which data is generated and processed (e.g., real-time analytics).

Variety: Different types of data – structured, unstructured, semi-structured.

Veracity: Trustworthiness and accuracy of the data.

Value: The insights and business value extracted from the data.

Challenges in Processing and Analyzing Big Data

- Scalability: Traditional systems cannot scale effectively with growing data.
- Data Storage: Difficulties in storing diverse and vast datasets.
- Data Integration: Need to process data from varied sources and formats.
- Real-Time Processing: Increasing demand for immediate insights.
- Security and Privacy: Ensuring data safety and compliance with regulations.
- High Costs: Infrastructure and skilled professionals are expensive.



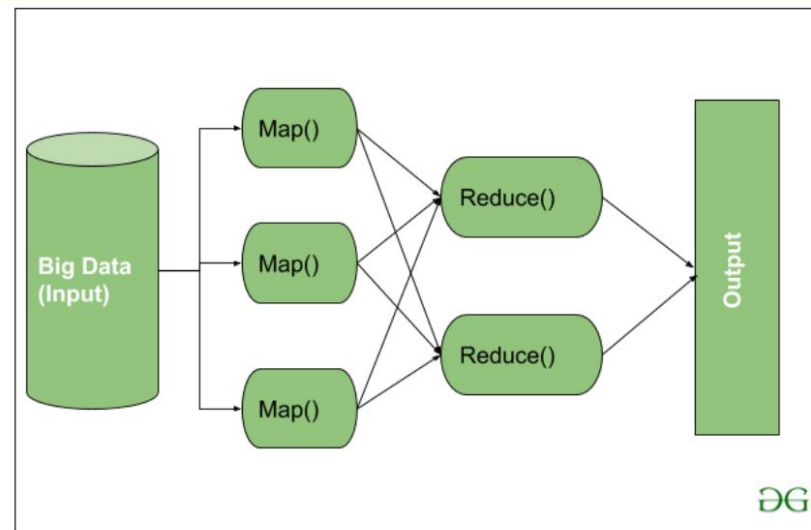
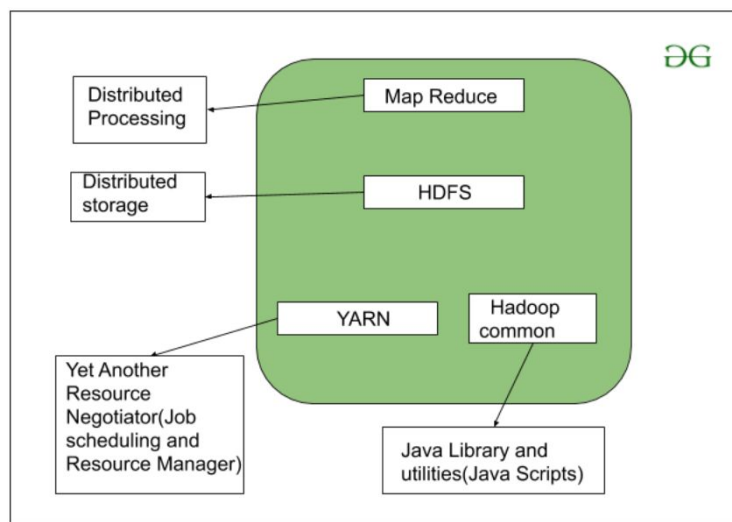
What is Hadoop



- Apache Hadoop is an open-source software framework developed for distributed storage and processing of large data sets.
- Created by Doug Cutting and Mike Cafarella.
- Inspired by Google's MapReduce and Google File System (GFS).
- Designed to scale up from single servers to thousands of machines.
- Hadoop uses a distributed computing model, breaking tasks into smaller units and processing them in parallel.
- Provides resilience through data replication.
- Offers horizontal scalability (add more machines to expand).
- Open-source and widely supported by a large community.

Key components of Hadoop

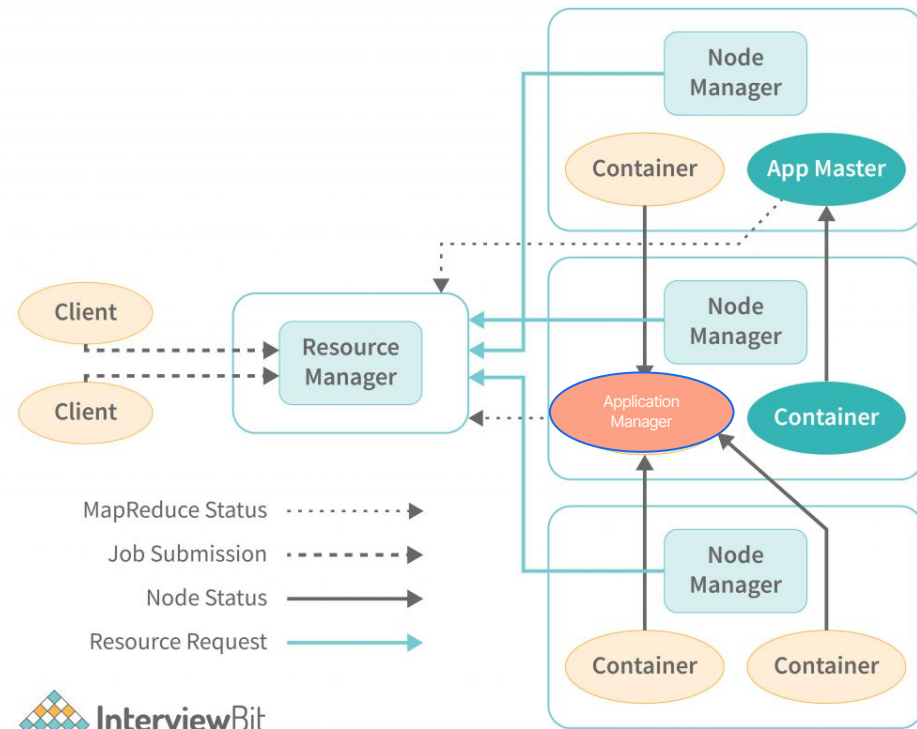
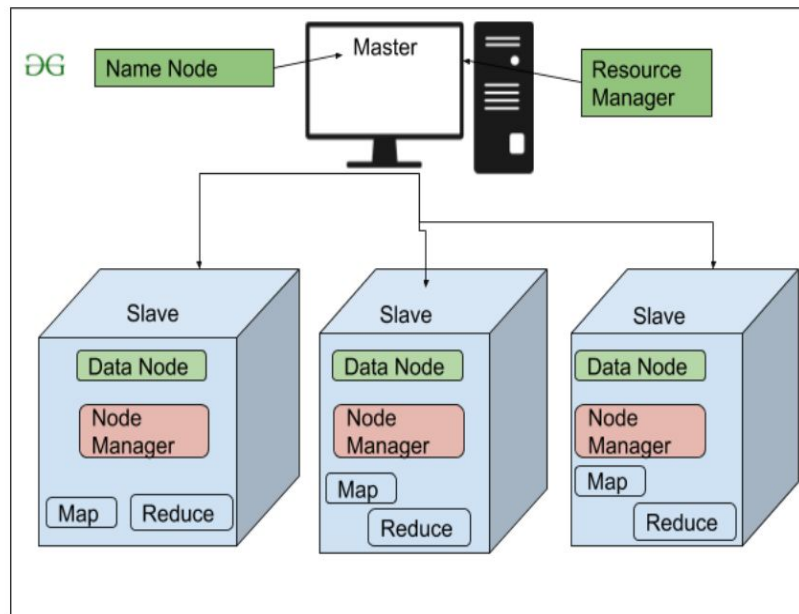
- **HDFS:** (Hadoop Distributed File System): Stores large files across multiple machines.
- **MapReduce:** Processes data in a distributed and parallel fashion.
- **YARN:** (Yet Another Resource Negotiator): Manages resources and job scheduling.
- **Common Utilities:** Libraries and APIs used by other Hadoop modules.



Hadoop Architecture

- Hadoop follows a Master-Slave architecture.
- Master Node: Manages metadata, task distribution, and resource allocation. Includes NameNode(HDFS), ResourceManager(Yarn)
- Slave Nodes: Store data and perform computation. Includes DataNode(HDFS), NodeManager, Container.
- Separates storage (HDFS) and processing (MapReduce).
- HDFS splits files into blocks and distributes them across cluster nodes.
- MapReduce processes data in parallel using the cluster.
- YARN schedules and manages computing resources dynamically.

Hadoop Architecture



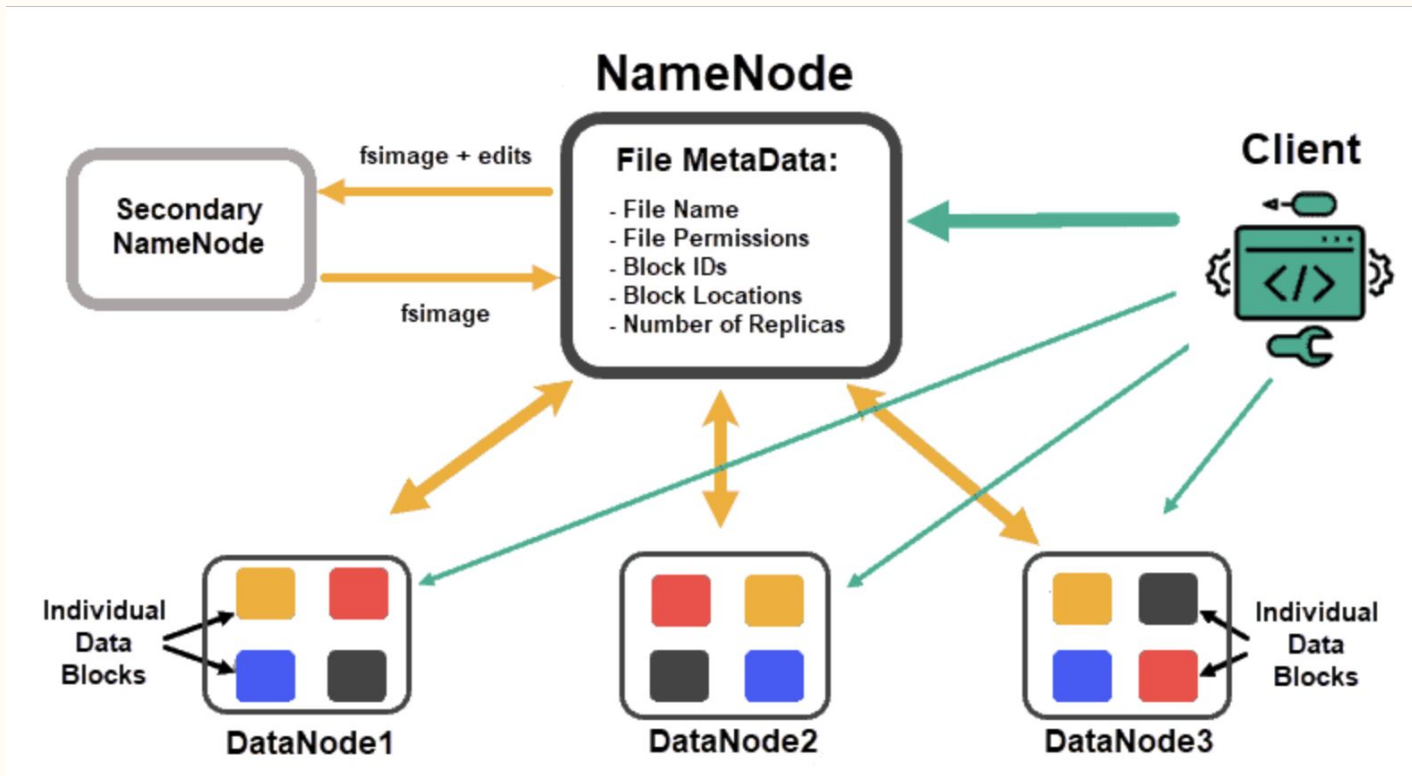
Explanation of Master-Slave Architecture

- **NameNode** (on Master Node, part of HDFS): Controls the file system namespace and regulates client access.
- **DataNode** (on Slave Node, part of HDFS): Responsible for storing and retrieving data blocks.
- **ResourceManager** (on Master Node, part of YARN): Manages resources in the cluster.
- **NodeManager** (on Slave Node, part of YARN): Manages execution of tasks on individual nodes.

HDFS (Hadoop Distributed File System)

- Designed to be highly fault-tolerant and to run on low-cost hardware.
- Optimized for large files and streaming data access.
- Allows data to be written once and read many times.
- Files are split into fixed-size blocks (default 128MB/256MB).
- Each block is stored on different DataNodes.
- The NameNode is the master server that manages the HDFS metadata. It tracks where all the data blocks are stored across all DataNodes in the cluster.
- Ensures efficient use of resources and high throughput.
- Default replication factor is 3.
- If one node fails, the data is still available from other nodes.
- Replication helps maintain system reliability and availability.

HDFS (Hadoop Distributed File System)

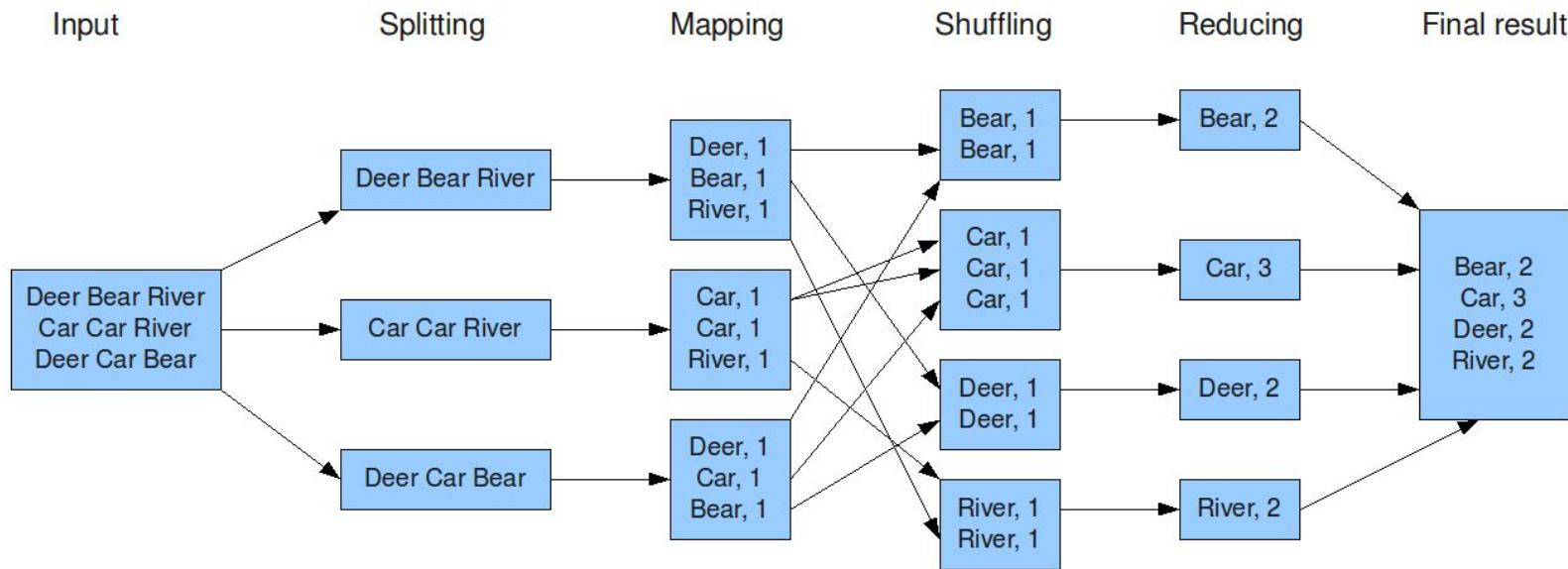


MapReduce

- MapReduce is a programming model to process large data sets with a parallel, distributed algorithm.
- Breaks processing into **Map** and **Reduce** phases. Handles failure automatically.
- Splits input data into chunks processed independently.
- Each chunk handled by different mapper functions.
- **Map**: Takes input and converts it into a set of intermediate key-value pairs.
- **Reduce**: Merges intermediate values associated with the same key.
- Example: Word count program.
- **Map Phase**: Splits input and processes it into intermediate data.
- **Shuffle and Sort**: Groups data with the same key.
- **Reduce Phase**: Aggregates results (e.g., sums, averages).
- Enhances scalability and performance.
- Reduces are also run in parallel after shuffle and sort.
- Enables high processing speeds for massive data sets.

MapReduce

The overall MapReduce word count process



YARN

YARN is the resource management layer for the Hadoop ecosystem. It is responsible for managing and scheduling resources in a Hadoop cluster.

Components of YARN:

ResourceManager (RM): ResourceManager is the central authority responsible for managing cluster resources. It allocates resources to running applications and ensures fair and efficient usage of cluster resources. It consists of two main components:

- **Scheduler:** Allocates resources to applications based on user-defined policies, such as capacity or fairness.
- **ApplicationManager:** Manages the lifecycle of applications, including tracking their status and application-level resource negotiation.

NodeManager (NM): Each node in the cluster runs a NodeManager, which is responsible for managing the resources on that node. The NodeManager monitors resource usage (CPU, memory, disk) on the node, handles the launching of containers, and reports the resource availability back to the ResourceManager.

ApplicationMaster (AM): Each application (such as a MapReduce job or a Spark job) has its own ApplicationMaster. It negotiates resources from the ResourceManager and works with the NodeManager to execute the application. The ApplicationMaster manages the execution of tasks within the containers.

Container: A container is a computational resource (like memory, CPU, etc.) that is allocated to an application. A NodeManager launches and manages containers, where tasks (such as MapReduce tasks) are executed.

Process Flow

1. Job Submission (User Initiates the Task)

- User submits a **MapReduce job** to the **YARN ResourceManager**.
- Job includes:
 - Input data path (in HDFS)
 - Map and Reduce logic (the actual program)
 - Output path (where to save the result)

2. Input Data in HDFS (Storage)

- Hadoop splits input data into **blocks** (default: 128MB).
- Blocks are stored across **DataNodes** (the slave nodes).
- **NameNode** (master) keeps track of where all these blocks live.

Process Flow

3. YARN Takes Over: Resource Management

- **YARN ResourceManager** receives the job.
- Starts an **ApplicationMaster (AM)** on one of the nodes. AM is job specific and temporary.
- AM requests **containers** (mini environments) from NodeManagers (on DataNodes).
- ➡ Parallelism: Multiple containers can be launched across nodes to run tasks simultaneously.**

4. Map Phase (Parallel Processing Starts)

- Each **Map task** processes one input split/block.
- These tasks are **assigned to containers** on nodes where the data is located (**data locality**).
- ➡ High Parallelism: Many Map tasks run **at the same time on different nodes**.

Example: 1TB file = 800 blocks → 800 Map tasks → Run in parallel across 100+ nodes.

Process Flow

5. Shuffle & Sort Phase

- After Map tasks finish, intermediate data (key-value pairs) is **shuffled** (grouped by key).
- Data is sorted and sent to the appropriate **Reduce tasks**.

6. Reduce Phase

- Each **Reduce task** processes a group of keys and aggregates their values.
- Run in separate containers (like Map tasks), in parallel

Process Flow

7. Output Written to HDFS

- Each Reduce task writes its result to **HDFS**, in its own output file (e.g., `part-r-00001`).
- NameNode manages where to store these, and DataNodes store the data.

8. Job Completion

- ApplicationMaster reports job success to ResourceManager.
- Temporary containers are released.
- User retrieves results from HDFS.

Hadoop Ecosystem

Hadoop ecosystem consists of various tools built on top of core Hadoop modules. These tools enhance data ingestion, analysis, querying, and coordination.

Overview of Various Hadoop Ecosystem Components:

- **Hive:** Data warehouse software with SQL-like querying.
- **Pig:** Scripting language for data flow.
- **HBase:** Column-oriented NoSQL database for real-time read/write.
- **Sqoop:** Transfers data between Hadoop and relational databases.
- **Flume:** Collects and moves large volumes of log data.
- **Oozie:** Workflow scheduler for managing Hadoop jobs.
- **Zookeeper:** Coordination service for distributed systems.
- **Mahout:** Machine learning library.

Benefits & Use cases

Benefits:

- Scalable: Easily expand storage and processing.
- Cost-Effective: Utilizes commodity hardware.
- Flexible: Handles structured and unstructured data.
- Reliable: Data replication and fault tolerance.
- Fast: Parallel processing leads to faster insights.

Use Cases:

- Social media analysis for sentiment and trend detection.
- Fraud detection in financial transactions.
- Customer behavior analytics for retail and e-commerce.
- Scientific research for large-scale data experiments.
- Healthcare analytics for patient records and diagnostics.
- Government analytics for population data, crime prediction.

Resources

Installation guideline for Ubuntu: <https://phoenixnap.com/kb/install-hadoop-ubuntu>

Hadoop Overview: <https://phoenixnap.com/kb/apache-hadoop-architecture-explained>

Simple Word Count: https://docs.google.com/document/d/1yvIjfDnBf_005S8IL4E4TMHGvJPiIXmVyd8cHjwZ0ZE/edit?usp=sharing

Simple Word Count (Apache Doc):

<https://hadoop.apache.org/docs/stable/hadoop-mapreduce-client/hadoop-mapreduce-client-core/MapReduceTutorial.html>



Thank you!